マップ

「渕　一博氏のロジックプログラミング」へのコメント

Comments on "Logic Programming by K. Fuchi"

前号（Vol. 9, No. 2, pp. 233-240）のAIマップは，いろいろと話題が豊富で，またAI研究分野にも多大な影響を与えた第五世代コンピュータプロジェクトの中心人物，渕　一博氏によるものでした．このプロジェクトについてはすでに数多く語られていますが，プロジェクトは氏の考え方，個性を強く反映したものであったことから，氏の個人的な気持ちを素直に語っていただいたことで，より内面的な理解が得られたのではないかと思います．プロジェクトの評価については国内外でさまざまな意見があり，当然氏の意見についても異論・反論があると思います．ここでは3氏のコメントを掲載させていただきます．　〔編集委員会AIマップワーキンググループ〕

1．雨宮真人氏（九州大学大学院総合理工学研究科教授）

1．ま　え　が　き

筆者は基本的に渕論文の主張と同じ立場に立つものであるが，筆者の問題意識は渕論文の主張と細かい点でいくぶん差があるので，このことについてコメントしてみたいと思う．それは，コンピュータ科学（特にAI）と論理，ならびに並列処理とロジックプログラミングの問題についてである．

筆者は，第五世代コンピュータプロジェクトの計画段階において，ロジックプログラミング，並列処理，知識処理を3本柱とする第五世代コンピュータの構想作りに参加し，またプロジェクト発足後は，並列処理の研究を通じてICOT組織の外からこのプロジェクトに間接的に関わってきた．最近では，定理証明系の手法をベースとしたロジックプログラミングの新しい展開の可能性に非常に興味を持っている．このような立場から，ロジックプログラミングにおける上記の問題についてコメントしてみたい．

以下の議論を展開するにあたって，まず「論理」の意味をはっきりさせておく必要がある．渕論文が指摘するとおり，コンピュータ科学において論理が重要であることは論を待たない．渕論文では論理を，コンピュータ科学の数学的基礎あるいは数理論理的枠組みという意味で広く（あるいは深く）捉えている．コンピュータはブール論理に従って決定論的に動作するし，プログラムの意味論にもいろいろな論理モデルが存在する．渕論文ではこれらの論理の基底となるものを記号的素過程あるいは論理的素過程と呼んでいる．筆者も，記号的素過程あるいは論理的素過程が（演繹，帰納）推論あるいは計算の基底，すなわちコンピュータ科学の根底をなすものであることに異論はない．しかし，論理をただちにロジックプログラミングと等価に置いてしまうことには多少のためらいを感じる．例えば，AIの問題をコンピュータ科学の枠組みで捉えるという立場に立って考えるとき，自然言語理解や画像理解などにおいて記号推論ですべてが律しきれるという点についてはそうだと主張できるほどの自信はない．また，ラムダ計算モデルやデータフローモデルをロジックプログラミングの枠内に含めてしまうことにもためらいを覚える．筆者は論理を「1階述語論理」のことだと狭く捉えているつもりもないし，反論理主義者でもないが，論理プログラミングや推論過程が議論の対象となる文脈においては，やはり述語論理という枠組みが意識されるのは仕方のないことではないかと考える．このようなわけで，以下では論理を述語論理という枠で捉えて議論することにする．

2. コンピュータ科学（AI）と論理

筆者と論理との関わりは，筆者が電電公社（現：NTT)の電気通信研究所で自然言語処理の研究を進めていた頃から始まる．通産省電子技術総合研究所推論研究室で自然言語処理の研究が行われていたのと同じ頃である．この研究のなかで述語論理をベースに意味解析・表現，推論の問題を研究していた．当時は使える言語は当然 Lisp であった．Planner や Conniver に接する機会もあった．この研究のなかで自然言語理解と会話の簡単なプロトタイプシステムを作ったが，その結果不満として残ったのは，規則的な推論過程として簡単には律しきれない自然言語という非常に複雑な対象を論理だけで処理できるのであろうかという疑問であった．

自然言語の処理には，構文と意味の一部という規則的に定式化できる部分と，文脈・状況などのようにまだ定式化が困難な部分とが混在している．そして，その規則的に定式化できる部分とは構文規則として整理できる部分であり，これは論理で処理できる．つまり，文法と述語論理（あるいは構文解析と推論）とは表裏一体の関係にある．渕論文の指摘する Earley や Pratt のパーサーとロジックプログラミングとの相性の良さは句構造文法と述語論理の形式とがその基本のところで等価であることによっている．しかし，問題は規則的な定式化が困難な部分，特に状況意味論（筆者の見解では現在の状況意味論はまだ満足のできる定式化ではない）や曖昧性解消の過程，さらには視覚情報と言語情報とのつながり，感情表現の扱いなどについて考えると，必ずしも記号を前提にした記号的素過程あるいは論理的素過程のみですべてを律するということは困難ではないかと思われる．実際，Winograd の SHRDLU の成功はこのような問題を捨象し，意味の理解が記号論的にまた手続き的に説明できる領域に限定したことにある（その後この方向の研究は数多く試みられたが，本質的に SHRDLU の枠を乗り越えたものは出現しなかったように思える)．かといって，ニューラルネットワークや遺伝アルゴリズムのような，「論理的素過程」がなくシステム構成の指導原理が欠如したモデルで解決できるとは思えない．

このような点からみると，渕論文における論理的素過程の議論には記憶機構とマッチング機構に関する視点が不十分なように思える．確かにロジックプログラミングの基底をなす単一化操作(unification)はマッチングの基礎を与えるものであるが，これは非常に固すぎる．柔軟な記憶機構とともにもっと柔らかなマッチングを許す機構が必要である．また，単一化操作は素過程を与えるものであっても，単一化操作に基づくロジックプログラミングでは柔らかな記憶の構造とそれに対する柔軟なマッチング機構は与えない．すなわち，柔軟な制御の機構は与えない．この意味で論理と非論理をつなぐ新しい枠組みの構築が極めて重要となる．これには，意味ネットワークやメモリベースの推論機構，並列マッチング機構などが重要なキーワードとなるが，論理的素過程がこれらの基礎を与えるモデルとなり得るかどうか．これは今後の研究に待たねばならない．

3. 並列処理とロジックプログラミング

渕論文では，関数型言語やデータフロー言語もロジックプログラミングの枠内に入るとしている．論理を広く捉えるならば，冒頭に述べたように，ロジックプログラミングがデータフロー言語や関数型言語を包含するという見方には同意できる．しかし，渕論文では直接触れていないが，両者の間の微妙な差は並列アーキテクチャや処理系を実現するうえでの問題点となって現れる．モデルの能力，一般性，美しさといった視点からは矮小化された議論だということかもしれないが，実現効率や性能など実際的な観点からはもっとシャープに区別する必要がある．処理系やコンピュータアーキテクチャの議論では，性能と効率はどうしても無視できない問題である．言語を議論するときは，記述性と効率という二つの相反する要求を常に念頭に置いておかねばならない．

筆者はかつてデータフローモデルをもとにロジックプログラミング向きの並列コンピュータのアーキテクチャやロジックプログラミング言語の処理系の並列化実現法を試みたことがあった．そして，ノイマンモデル，データフローモデル，リダクションモデルのいずれを問わず，現状のアーキテクチャではどうしても解決できない問題に遭遇した．AND 並列と OR 並列の両面でロジックプログラムの並列化を試みたわけであるが，このとき，単一化とデータフローの整合性の問題と，AND 並列-OR 並列同時実現における変数束縛(実行環境）管理の問題がネックとなった．

データフローとの整合性の問題は，単一化操作を基本とするロジックプログラムではデータフローの方向性は実行するまでは決定できないということである(データフロー方向の非決定性)．このことはロジックプログラミングの大きな特徴であり，ここに記述能力の高さを示す重要なポイントがある．しかし，この記述性の高さの代償として実現コストが大きなものとな

ってしまう．そのとき考えた解法は，各メモリセルに制御ビットを設け，あるプロセス（Producer）による変数束縛が生じたとき，実行を中断している他のプロセス（Consumers）の実行を駆動するというものであった．この原理は，データフローアーキテクチャで用いるI-structureや要求駆動のアイディアにヒントを得たものであったが，これは結構オーバヘッドをくうものであった．特に第2の環境管理の問題と絡んでくると，並列アーキテクチャの実現コストは非現実的なものとなってしまう．

第2のAND並列-OR並列における実行環境管理の問題であるが，OR並列はいくつかの実行が同時並行的に行われ，その数だけ並行プロセスが生まれる．まだ，AND並列ではAND並列プロセスにまたがる変数束縛の整合性チェックなど実行環境の管理にオーバヘッドをくうことになる．さらにAND並列-OR並列を同時に実現させようとすると，AND並列プロセスとOR並列プロセスとが交互に混ざり，これらの間での変数束縛の管理は大変なオーバヘッドとなり，実用には耐えない．この点，データフロー言語や関数型言語の並列マシン上への処理系実現はかなりすっきりしており，実用的である．

ロジックプログラミング言語に特に制限を加えることなくAND-OR並列を実現することはほとんど困難だと思っていた頃，ParlogやConcurrent Prologが提案され，続いてICOTからGHCが提案された．これら一連の言語に共通している点は，全解探索を指向するDon't know nondeterminismの記述に制限を加え，Guarded commandの概念に立つCommit choice型のDon't care nondeterminismによる並列処理記述に焦点を絞ったことである．これには上記のような並列実装上の問題点を言語機能の制限によって解決することに加えて，ロジックプログラミングでOSのような実用的なシステム記述ができるのかという批判に答えることも一面にはあったように思われる．当時筆者が悩んでいた問題はこのような言語の制限によってある程度解決された．しかし，関数型言語・データフロー言語の分野ではすでにストリーム処理や非決定性処理の研究が進んでおり，筆者の立場から見れば，このことはロジックプログラミングが関数型言語の影響を受けた結果のように思えた．特に，GHCについては本質的にデータフロー言語への回帰ではないかとの印象を強く持った．

第五世代プロジェクトが始まった頃は，データフロー言語や関数型言語の処理系に関してはまだナイーブな実現方式の研究段階であり，まだ実用にはほど遠かった．しかし，この後の研究の進展により，データフロー解析によるコンパイラ構成技術とスレッド処理アーキテクチャが開拓されてRISCプロセッサ方式との距離もしだいに縮まってきた．現在，細粒度並列処理の技術は実用化可能な領域に入りつつあるといえる．

ロジックプログラミング言語とデータフロー・関数型言語の関係については立場によってそれぞれの主張があるのは当然であろう．しかしそれより重要なことは，ロジックプログラミングにせよデータフロー・関数型プログラミングにせよ，宣言型の細粒度超並列処理指向のソフトウェアパラダイムが超並列処理環境の実用化に向けてその重要な基礎を占めるようになってきたということである．

4. あ と が き

第五世代プロジェクトの10年間で世界のコンピュータ科学・技術は大きく変化した．プロジェクトの計画段階で考えていた技術のあるものは予測どおりに実用化され現実のものとなった．これらのなかで顕著なものは，コンピュータのダウンサイジング化とワークステーションおよびネットワーク環境である．またさらに大きな変化は，プロジェクトの計画時には世間一般には否定的な見方が強かった超並列コンピュータの開発である．超並列コンピュータに関してはいまや商用マシンが出現し，超並列ソフトウェアも徐々に開発されてきている．しかし，この10年間で期待どおりに進展しなかったものも多い．知識処理や自然言語などAIの実用化はまだまだの感がある（エキスパートシステムについてはむしろ期待が大きすぎたと言ったほうが，妥当かもしれない）．むしろAIに関しては，さらに基礎研究を積み上げることの必要性がより明確に認識されてきているといえる．

第五世代プロジェクトについて見れば，その成果が直接実用技術に結びついたものは少ないかもしれない．しかし，渕論文が主張するように，もともとこのプロジェクトは論理を基礎とするコンピュータ科学の創世という文化の変革を目指したものであり，かなりの基礎研究指向であったことを考えると，その効果は十分得られたと考える．この10年の間に研究の種をまき，10年たった今その種が芽を出し，それぞれ本格的な研究が始まろうとしていると見て取れる．具体的なキーワードをあげれば，超並列アーキテクチャと超並列言語・ソフトウェア，定理証明システムや制約プログラミング，演繹・帰納推論，学習などのロジックプログラミングをベースとする新しい研究展開，大規模知識ベースと自然言語処理，などなどである．これら

に加えてAIという観点からさらに新たな問題を提起するならば，論理と非論理の調和を目指す枠組みの構築を目指す研究の重要性である．この観点からロジックプログラミングがその論理的素過程となるかどうか興味のあるところである．

2．柳生孝昭氏（日本ユニシス(株)常務取締役）

渕氏の「AIマップ」の主部である第五世代計画に対して，筆者はいわば場外観客に終始した．そこでまず，事情通には常識かもしれないが，いくつかの基本的な事柄に関する，初等的な質問をさせていただく．次に実世界と情報の世界との関連，特にdata modelingをめぐる筆者の立場を述べる．最後に，その立場からQuixoteについて所見を加えたい．

1．いくつかの，初等的な質問

（1） 何が論理でないか，広い意味の論理とは何か？

広い意味で，という条件つきではあるが，関数型も対象指向もLPのうちとされる（3•1節（参照は渕論文，以下同様））．では，LispやC＋＋はいかなる意味でLPであるのか？ 逆に，いかなる言語が広義のLPにも数えられないのか，それはなぜか？ そもそも論理と非論理を分けるのは何か？

（2） 選んだのはLPの考えか，Prologか？

Kowalskiから学ぶLPの考え（2•3節）とはProlog流にほかならない．効率や既存の資産が問題になるのも早い話がProlog対Lispであって，後者を含む広義のLPと，それ以外の比較ではあるまい．とすれば，これらの問題を承知のうえで選んだのは，LPの考えではなく，端的にPrologではなかったのか？ あるいはLPの考えといっても，広義ではなく，狭義のそれではなかったのか？ 思うに，論争を不毛にするのは，その神学的であるよりも，言葉の曖昧さ・多義的な用法なのであろう．

（3） 形式理論 対 解釈

論理(的接近法の定着)を数学(的基礎の確立)と言い換えてもよい，とされる．また，Coddのmodelは述語論理に基づくといわれる(3•1節)．本当か？ Gödel以後，論理（形式化された理論）と数学の別は常識ではないのか？ Schemaを形式理論と見ることはできるが，instanceの集まりとしてのdatabaseは，伝統的に解釈とみなされてきた．DBを単位節の集合と捉えるのはLPに触発された伝統との決別であろう．両者を分けるのは重要である．いずれの立場を取るかによって，例えば整合性条件の扱いも異なるからである．この区別を無視し，さまざまな立場を「論理的」と一括して形容するのは，多様な事情の過度の単純化，または「論理」という言葉の意味の過度の拡大ではないか？ 問題は，特定の立場に固執する「純枠主義」の是非ではなく，言葉の適切な使用に関することではないのか？

（4） 操作（program）と操作対象（database）

ProgramとDBは一つの世界の二つの側面である，という(3•1節)．それぞれの側面はそれ自身，対象と対象相互の働きかけからなる，全体世界から抽象された，一つの小世界であろう．例えば，工業製品の機能(形，挙動など）や管理（価格，在庫量など）の面のように．側面は独立性が高く，それでいて全体世界という源泉を共有する．ProgramとDBの関係はこれと全く違う，互いに異質で，しかも独立に存在し得ない，異質なるがゆえに結びあって，世界という複合体を構成し得るような関係ではないか？ Sub-system(小世界）になり得るのと，(論理という方法的）基盤を共有し得るのとは，本質的に違うのではないか？

（5） 演繹，発想および帰納

これらの基底に共通の論理的素過程がある，とされる（5•3節）．その意味は，例えば演繹 $H \wedge T \Rightarrow C$ を $\neg C \wedge T \Rightarrow \neg H$ と書き直せばすなわち発想となる，さらに T が節形式 $H_i \rightarrow C_i$ からなるならば，LPの算法を等しく適用できる，ということか？ しかし，T が非Horn節を含み，推論結果の H が否定形を含む場合，仮説としての説明力を欠く，といえないか？ またN. Hanson流に $H \wedge T \wedge T' \Rightarrow C$（$T'$ は既知の理論）を考えるとき，仮説 H と新しい理論 T は説明すべき C を陽に含まないかもしれない．発想には演繹との共通性を超える含蓄，既知の理論の選択と，新しい理論の創造があるのではないか？ それらに対してもLPの方法が適用できる，と主張されているのか？ LPの枠内で扱い得る限りで発想を捉えたうえで，発想もLPで扱い得るという自明の理を説いてはいない

か？

（6） いかなる「現実」に目を向けるのか？

C言語の拡張への現実的な期待にもかかわらず，CLPやPLPが有力視されている（5•1節）．ここでの「現実」とはいかなる現実か？ 偏見にとらわれたAI研究の場を指すのか？ それとも量産されたCobol programmerの同業組合のことか？ 後者であるならば，（逐次，制約，並列を問わず）LPが広まる条件が整っている，または整いつつある，と考えられるのか？ 否であるならば，何をなすべきか？ そうしてそれは良いことなのか？ 何の，誰のために？

2.「もの」，概念（数学）および記号（論理）

Programやdataはまずは記号である．人はそれに意味を与え，概念として読む．概念は人の「もの」の世界（実世界）の理解を表す．「もの」，概念，記号の三者を峻別することが，情報処理の営みの全的な了解のために必須である．

（1）「もの」と概念

「もの」とは物理的存在・知覚の対象を指す．すべての情報systemは何らかの仕方で「もの」に関わる．とはいえ，programやDBが常に直接「もの」の概念化であるわけではない．典型的な事務処理は機械化以前に，すでに「もの」を離れた抽象概念の世界を成している．Data modelの意味論や対象指向の方法は，改めて情報科学が人の概念形成の次元を考察することを促した．ここで注意すべきは，第1に（当り前のことだが）概念が計算機上で有意味なほどに厳密に定義されること，第2にいかなる概念体系も「もの」の世界の意味を尽くせない，したがって概念の開かれた（open-endedな）記述が保証されること，である．

（2）「もの」の意味，「もの」と「こと」

「もの」は属性の集まりではない．まして属性の一部である「本質」によって，同定をされない．「もの」の意味は，そのものを見つめていてもわからない，他の「もの」との関係＝「こと」の総体によって定まる．一粒のちりのうちにも森羅万象が映し出される（華厳経），逆に無色の対象の本質は，事実の構成要素となることにある（L. Wittgenstein）のである．「もの」を属性の組として捉える古典的な関係modelや，「こと」を「もの」に従属させる対象指向の方法は，正しいか？むしろ「もの」を脱意味化された記号として表し，「こと」に第1級市民の地位を与えなければなるまい．

（3） 概念と記号，または数学と論理

概念の世界としての情報systemは，広い意味の数学的体系である．論理的言語で記述する場合もしたがって，単一化に基づく推論だけでなく，数学の通常の方法が広く，自由に使えてしかるべきだろう．特に，手続きと集合算を含む関数と論理式の評価を許さねばならない．これはDBを形式理論ではなく，解釈と見ることを意味する．その一般的な状態は公理（schema）の一部を満たしている．完全に満たすべくDBを発展させる動的な相が，もっと注目されてよい．そこで働く知はknow-whatやknow-thatではなく，'know-how'と呼ばれるべきものであろう．

3. Quixoteの世界について

対象指向の概念の多くがさまざまな分野に由来することは周知である．それはOOの特性が理論面よりも実際的体系化にあることを示す．Quixoteはさらに CLPや状況の概念を加え鮮やかに料理した，と思う．だが，それだけに概念のはらむ困難を際立たせたことも否み難い．

（1） OO一般についていえることだが，構造の有向graphへの限定が関係の対称性を犠牲にしている．例えば，$X/[\text{common_friend}[\text{with}=Y]\rightarrow\{Z\}]$（森田，ほか：Quixoteのオブジェクト識別性，情処研，90-AI-73）は，$\text{common_friend}(X, Y)\rightarrow\{Z\}$と書くほうが簡明で，自然ではないのか．

（2） 属性を本質的（intrinsic）と偶有的（extrinsic）に分けるのには，本当に驚いた．アリストテレスに帰るのならばしらず，かかる区別はすでにライプニッツが放棄したのではないのか．例えば「点」の本質とは何なのか？「位置のみがあり，広がりがない」というわけにもいくまい．座標値によるにしても，座標系をどう示すか？ 動きつつも同一性を保つ場合はどうするか？ 線や面も同様．要素の定義は，ヒルベルトの精神に従い，互いの関係の総体として与えるほかないではないのか．

（3） OOと演繹DBの結合の結果，対象の存在の表明と性質の記述という次元の異なる事柄が，原子論理式としてのobjectまたはattribute termのなかに同居することになった（神の存在論的証明を想起させる！）．端的に，property spec.を欠くbasic objectは，論理式としていかなる資格を持つのか．

（4） これも初等的な問いの類になるが

$$\text{john}/[\text{son}\leftarrow\{X\}]\ (\text{john. son}\supseteq_H\{X\})$$

はjohn. son＝{male/[age＝3]}とX＝male[age＝20]から導かれる．しかし，そんなはずがあるまい，どこで間違ったのか？ それになぜ$\text{john}/[\text{son}\leftarrow X]$と書けないのか，$\text{john}/[\text{age}\rightarrow\text{integer}]$とは書けるのに？ john. sonとjohn.ageの存在論的な身分はどう違うの

か？　思うに，Quixoteは(明示してはいないが)ある特定の存在論・世界観に基づいているのであろう．それはどのような「実世界」を想定しているのであろうか．

3．林　弘氏（(株)富士通研究所 マルチメディアシステム研究所システム研究部門長）

渕　一博氏のAIマップでは，ロジックプログラミングに関する歴史，意義，第五世代プロジェクトおよびその成果について述べられている．ここでは第五世代プロジェクトについて民間の研究者の立場から意義，評価をコメントさせていただく．

1．第五世代プロジェクトについて

ロジックプログラミングを中心に置いた第五世代プロジェクトは1982年より開始され，1994年度で終了する．この間幾多の研究者の参加により，過去の日本のプロジェクトでは比類をみないほど海外からも注目を浴びてきた．

渕論文では，第五世代プロジェクトが共通基盤とした「論理」の歴史と第五世代プロジェクトの研究における作業仮説として考えられた「意味論」および「並列推論」の位置づけが示されている．しかし，第五世代プロジェクトに対してのマスコミを中心とした外部の世界は，プロジェクトの研究的側面より，プロジェクトが目的とするあるいは狙っている新しいコンピュータの新しい機能の実現に期待した．

この頃は日本の経済力が世界的にも恐れられるようになった時期であり，新しい情報処理の分野でも日本が世界を席巻するのではないかという危惧から，欧米でも似たようなプロジェクトが続々と開始された．米国のMCC，英国のAlvey，欧州のESPRITなど，新しいコンピュータあるいは新しい情報処理を目指したプロジェクトが各国の科学政策のポイントとなった．

この点から，ロジックプログラミングを中心に置いた第五世代プロジェクトは世界に大きなインパクトを与えてきたといえる．

2．第五世代プロジェクトとAI

第五世代プロジェクトの共通基盤はロジックプログラミングであったが，外部から見ればAI技術そのもの研究プロジェクトと理解された．つまり，第五世代プロジェクトによって，AI技術をベースとした新しいコンピュータが開発されると期待された．日本ではこの頃から「AIフィーバ」がマスコミで取り上げられ，ビジネス分野でも多数のアプリケーションが試作開発されるようになった．AI技術が注目を浴びたのは，第五世代プロジェクトの影響によるところが，大であった．

また一方で，第五世代プロジェクトが開始された1980年代の前半はワークステーションの勃興期でもあった．SUNあるいはAPOLLOなどが汎用ワークステーションの代表であり，SymbolicsやLMIがAIワークステーションの代表であった．これらのAIワークステーションは，米国政府のプロジェクトのなかで開発されたLISPマシンをベースに製品化され，高度な表示機能あるいはその操作性から，当初は汎用ワークステーションに比べ各方面の注目を浴び，産業界においても多数導入された．

第五世代プロジェクトの前期では，PSIワークステーションが試作設計された．このシステムはPrologを拡張したESPを機械語とした専用マシンであり，ESPのみで書かれたOS SIMPOSが動作した．SIMPOSは論理型言語による大規模ソフトの世界最初の実験としては十分評価された．

しかし，PSIシステムは研究分野を中心にようやく広まってきた汎用のワークステーションと比較されることが多かった．「なぜLispでなく，Prologか」，あるいは「なぜUNIXワークステーションでないのか」，など．

この頃，世の中ではAIワークステーションに比べ汎用ワークステーションの評価が高まっていた．汎用ワークステーションの低価格性とソフトウェア開発環境の良さが認識されてきた．また，AI分野における高度な機能実現が非常に困難であることが再認識されるようになってきた時期に符合しているのも，このような意見が出てきた一つの背景であった．

第五世代プロジェクトのPSIは，渕論文で述べられている「Prologは出発点にすぎず，私たちが採用したのは，ロジックプログラミングという“考え方”」とのいう主張を示す格好の例として評価されるべきものであった．しかし，AI技術の進歩が世の期待に比べて遅かったために，世の中の興味が他の分野へ移ってしま

ったのは，タイミング的にまことに残念であった．もし5年早くPSIが実現していれば，評価は随分と違ったものになったと推測される．

3．第五世代プロジェクトと並列推論

第五世代プロジェクトの中期，後期は論理型言語による並列推論の研究が中心であった．GHC/KL1は渕論文にあるように，Horn論理を出発点にして，これに並列動作の制御のために，ある「制約」を加えたものであり，当初一部にはいろいろ批判があった．しかし，これによって同期の機能が実に自然に組み込まれており，他の並列言語に比べ評価される点は多い．

後期には並列推論マシンPIMも完成し，このうえで種々のアプリケーションが開発された．最近の成果では，知識処理分野においてもデータの並列性が性能向上のキーファクタであることが示されている．

また，KL1の特徴はアプリケーションプログラム内の同期を開発者が意識しなくてもよいことから，最近の研究成果として並列アルゴリズムの開発とプロセッサ間の負荷分散を独立して制御できることが示された．プログラム開発者は，最初逐次マシン上で並列アルゴリズムを開発しデバッグを済ませる．その後に並列マシン上で効率的なプロセッサ間の負荷分散を実現する．

通常の並列マシンでのプログラム開発は，アルゴリズムの開発と効率的な負荷分散の実現を同時に行う．このためのプログラム開発は優れたソフトウェアの開発環境を使用したとしても，非常に困難となっている．

これらの成果は，KL1言語が並列処理アプリケーションを開発する際に非常に優れた特徴を持っていることを示している．

したがって，並列推論の研究などは第五世代プロジェクトの研究的な側面から見ると非常に成果を上げたと評価される．

4．第五世代プロジェクトと実用化研究

第五世代プロジェクトでは，新しいコンピュータの新しい機能を実現することを世の中が期待してきた．いわゆる知識処理あるいはAI技術の実現である．並列推論の研究もこの延長線上にあった．

しかし，**2章**で述べたように，AI技術の研究が期待ほど進歩しなかったため，世の中の並列研究あるいは実用化の大勢が知識処理の分野とは違った分野に向き始めたことが，当初のシナリオを難しいものにした．

具体的には，汎用マシンのソフトウェア開発環境のもとで，従来からあるソフトウェアを並列に動作させ，高速化を実現することが中心となってきた．数値処理を含め，既存のソフトウェアのアルゴリズムがいろいろな並列マシン上で実現されようとしている．

並列処理の研究のみでなく，一般に研究開発環境が専用マシンあるいは専用ソフトウェア開発環境であれば，それを使える研究者のグループも限られた範囲となり，研究の発展もおのずと限界がある．

第五世代プロジェクトの後継として運営されている2年間の「研究基盤化プロジェクト」においても，上記の状況を踏まえ，研究開発の中心は汎用マシンに移ってきた．UNIXワークステーションあるいはパソコン上で動作するKL1言語の開発が大きな比重を占めている．KL1言語がUNIXワークステーション上で稼働すれば，世界中の研究者がこの言語を活用でき，相互の研究評価がより明確にできる．研究開発環境の標準化が研究開発のポイントであることが明確にされてきたことは，第五世代プロジェクトの大きな知見といえる．

1980年代のAI技術実用化開始段階では，AI技術はそれだけで一つの世界を作り得るとの考え方が一般的であった．AIワークステーションはその代表的な例であった．AIシステムをそのマシンの環境上ですべて作り上げる．しかし，開発コストあるいは技術の普及性から，そのような環境でシステムを構築するより，汎用的な技術上にシステムを構築していくことのメリットが認識されてきた．AI技術のみですべてのシステムを作り上げるのではなく，AI技術を一つの技術として活用し，他の技術と連動することによって全体的に優れたシステムが構築される．第五世代プロジェクトの成果もそのような環境で活用することによって初めて生きてくる．

5．第五世代プロジェクトと世の中へのインパクト

第五世代プロジェクトが開始された頃の熱狂と10年経過した現在の状況を見たとき，結局世の中にいかにインパクトを与えたかが評価の大きなポイントとなっているように見受けられる．

研究面から見たとき，このプロジェクトは非常に大きな成果を上げてきた．ロジックプログラミングの研究への貢献は十分評価される．しかし，現実の情報処理の世界への適用はまだまだ未熟である．すでに何度も述べているが，AI技術の進歩が世の中の期待からするとまだ不十分のため，ロジックプログラミング技術の現実世界への適応も不十分となっている．

一方，この10年間の情報処理の世界はプロセッサ，

OS，ネットワーク，ダウンサイジングなどの面から，予想以上に急激であり，世の中へのインパクトは強烈なものとなっている．ここに，ロジックプログラミングあるいはAIというキーワードは見受けられない．

第五世代プロジェクトのような国家プロジェクトでは，たとえ研究を対象としたプロジェクトでも，世の中は実世界へのインパクトを期待している．実世界における成果の普及度がプロジェクトへの注目度を測る大きな要素となっている．

結局，「第五世代プロジェクトは，ロジックプログラミングあるいは並列推論などの研究分野では大きな成果を上げたが，実世界へのインパクトがまだまだ不足している」とコメントできる．この点は，最近の日本の状況を見ると，単に第五世代プロジェクトの課題といえるだけでなく，日本のコンピュータに関連する学会，官庁，産業界各方面の課題といえる．